Jaypee Institute of Information Technology, Noida

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AND INFORMATION TECHNOLOGY



Project Title: RenderLab, a software-rasterizer graphics laboratory

Enrol. No.	Name of Student
9923103059	Yasharth Singh
9923102028	Rakshit Suneja

Course Name: Geometric Algebra for Computer Graphics

Course Code: 22B12MA412

Program: B. Tech. CSE

Semester VII

2026 - 2027 (odd semester)

ACKNOWLEDGEMENT

We would like to place on record our sincere thanks to Dr. Ajay Kumar Gupta, course coordinator for Geometric Algebra for Computer Graphics (22B12MA412), Jaypee Institute of Information Technology, Noida, for the course that gave this project its topics and for the guidance given while we worked on it.

We also thank our classmates for trying the labs and for their comments on how the screens could be made clearer.

ABSTRACT

RenderLab is a program that runs in a web browser and helps a student see how computer graphics works. Every picture on the screen is drawn by our own code. The code writes colours into an array of pixels and the browser shows that array. We did not use WebGL or any 3D library.

The project is made for the course Geometric Algebra for Computer Graphics. It has the four algorithms the course asks for: Bresenham line drawing, back-face culling, alpha compositing (the Porter–Duff "over" rule) and Visvalingam–Whyatt line simplification. Each one has its own lab. The lab shows the formulas, the numbers at each step, and a comparison with a simpler or wrong version. Other labs cover 2D and 3D transformations, determinants, quaternions against Euler angles, the outer product, the sine rule, line intersection, bases and handedness. As extras, there is a demo scene with a small scene editor, and a benchmark tab that times our code.

All 487 automated tests pass. We tried all 195 controls by hand and fixed two layout bugs. In the benchmark (15 timed runs after 5 warm-up runs), back-face culling removed 50.33% of the 15,412 triangles of the demo scene and cut the time for one frame from 41.4 ms to 28.0 ms. The older DDA line method was faster than Bresenham in JavaScript (68 ms against 122.5 ms for 10,000 long lines). Visvalingam–Whyatt took 14.3 ms for 2,000 points, and its time grew roughly with the square of the number of points.

Keywords: software rasterizer, Bresenham, back-face culling, alpha compositing, Visvalingam–Whyatt, quaternion, outer product.

Table of Contents

Page No.

Acknowledgement . . . . . *i*

Abstract . . . . . *ii*

List of Tables . . . . . *v*

List of Figures . . . . . *vi*

List of Abbreviations . . . . . *vii*

Chapter 1: INTRODUCTION

1.1 General 1

1.2 Motivation 1

1.3 Objectives 1

1.4 Scope 2

Chapter 2: MATHEMATICAL BACKGROUND

2.1 Outer Product and Area 3

2.2 Transformations and Determinants 3

2.3 Quaternions 3

2.4 Bases and Handedness 3

Chapter 3: DESIGN AND ARCHITECTURE

3.1 Tools 4

3.2 Two Layers 4

3.3 Drawing Steps for One Frame 4

3.4 The Editor Window 4

Chapter 4: THE FOUR REQUIRED ALGORITHMS

4.1 Bresenham Line Drawing 5

4.2 Back-Face Culling 6

4.3 Alpha Compositing 6

4.4 Visvalingam–Whyatt Simplification 7

Chapter 5: THE OTHER LABS AND THE REACTOR SCENE

5.1 Transforms Lab 9

5.2 Quaternion Lab 9

5.3 Geometry Lab 9

5.4 Reactor Scene and Scene Editor 10

Chapter 6: TESTING

Chapter 7: EXPERIMENTAL RESULTS

7.1 How We Measured 12

7.2 Back-Face Culling 12

7.3 Bresenham against DDA 12

7.4 Alpha Compositing 13

7.5 Visvalingam–Whyatt against RDP 13
7.6 Transformations 13

Chapter 8: LIMITATIONS, CONCLUSION AND FUTURE WORK

8.1 Limitations 14
8.2 Conclusion 14
8.3 Future Work 14

References **15**

List of Tables

Table	Title	Page
4.1	Bresenham steps for the line $(0, 0) \rightarrow (5, 2)$	5
6.1	Tests by area	11
7.1	Frame time with culling off and on (median of 15 runs, ms)	12
7.2	Time for 10,000 lines (median, ms)	12
7.3	Simplifying to 10% of the points (median ms per call)	13
8.1	Contribution of each student	14

List of Figures

Figure	Title	Page
1.1	The RenderLab window with the Rotating Cube open	1
3.1	The two layers of RenderLab and the path of one frame	4
4.1	The decision in `bresenhamSteps` (`src/core/render/line.ts`)	5
4.2	Bresenham Lab. Left: a line half drawn with its table of error values. Right: the DDA line drawn over it.	5
4.3	Culling Lab, wireframe with normals. Left: culling on. Right: culling off, so the back faces are drawn too.	6
4.4	The core of `over` (`src/core/render/alpha.ts`)	6
4.5	Alpha Lab in 3D. Left: panes sorted back to front. Right: sorting off, so the colours are wrong.	7
4.6	The scan and the ranking rule (`src/core/simplify/visvalingam.ts`)	7
4.7	Simplify Lab. Left: 500 points against 75, with the RDP result drawn over it. Right: step mode, one removal at a time.	8
5.1	Transforms Lab. Left: 2D, with the F shape, basis arrows and determinant. Right: 3D.	9
5.2	Quaternion Lab. Left: gimbal lock at pitch 90°. Right: slerp against lerp with the plot.	9
5.3	Geometry Lab. Left: outer product mode. Right: bases in the plane with Gram–Schmidt.	10
5.4	Reactor Scene. Left: one object selected and outlined. Right: the same scene as a wireframe.	10
7.1	Charts saved from the Benchmark tab. Left: frame time with culling off and on. Right: time to simplify, VW against RDP.	13

List of Abbreviations

CPU	Central Processing Unit
CSV	Comma-Separated Values (a plain table file)
DDA	Digital Differential Analyzer (a simple line-drawing method)
NDC	Normalized Device Coordinates (the cube -1 to 1 the camera maps the scene into)
RDP	Ramer–Douglas–Peucker (a line simplification algorithm)
RGBA	Red, Green, Blue, Alpha
SIGGRAPH	Special Interest Group on Computer Graphics and Interactive Techniques
UI	User Interface
VW	Visvalingam–Whyatt (a line simplification algorithm)

Chapter 1: INTRODUCTION

1.1 General

Most graphics tools hide how a picture is made. A library such as WebGL takes triangles and gives back a picture, and the student never sees how one pixel was chosen. RenderLab shows these steps. It is a small renderer where every step is our own code, so each step can be paused and checked.

The program runs in a web browser. One frame is a plain array of pixel values (a `Uint8ClampedArray`). Our code fills it and the browser copies it to a canvas. The window looks like a game-engine editor (Figure 1.1). The labs are listed on the left. The picture is in the middle, with an Output panel below it that prints what the algorithm is doing. The controls and formulas are on the right.

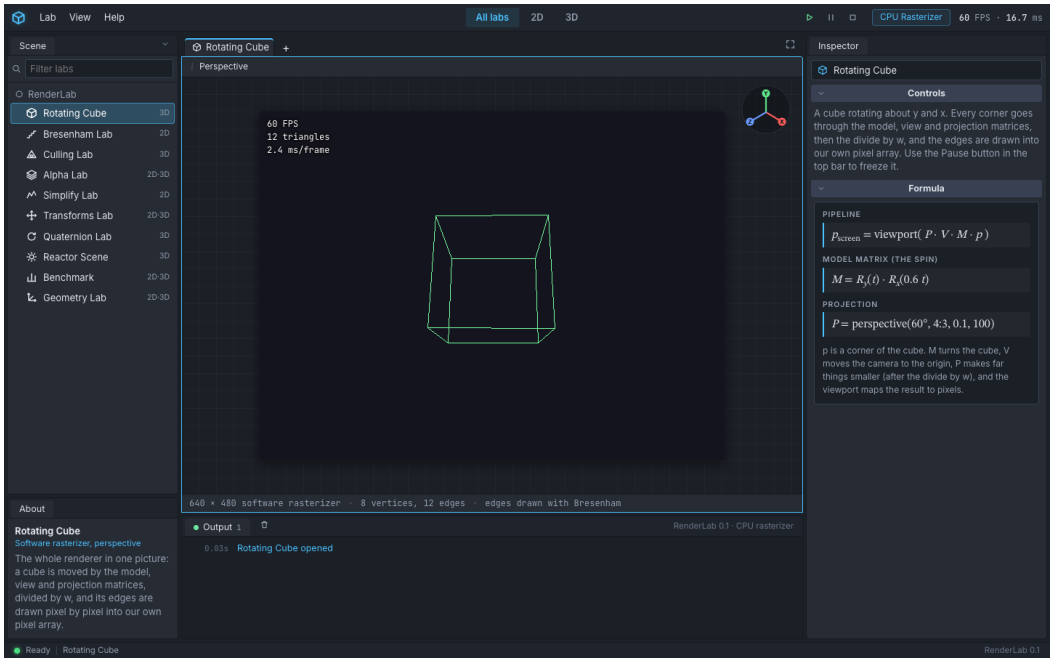


Figure 1.1: The RenderLab window with the Rotating Cube open

1.2 Motivation

The course Geometric Algebra for Computer Graphics (22B12MA412) has ideas that are easy to read but easy to get wrong in code. Examples are the error value in Bresenham's line, the sign of a cross product in culling, the order of layers in a blend, and the order in which points are removed in line simplification. We wanted a tool where changing one number changes the picture and the working next to it. We also have to explain every line of our code, so we wrote the renderer ourselves in plain TypeScript with comments.

1.3 Objectives

- Write the four required algorithms by hand: Bresenham line drawing, back-face culling, alpha compositing and Visvalingam–Whyatt.
- Give each algorithm a lab with formulas, step-by-step numbers and a comparison.
- Add labs for the other course topics: transformations, determinants, quaternions, the outer product, bases and handedness.
- Keep the algorithm code apart from the user interface, and test every algorithm.
- Measure our own code and report the numbers as they came out.

1.4 Scope

The four required algorithms and their labs are the main part of the project. Two things are extras: the Reactor Scene (a demo scene with a scene editor, lighting, fog and bloom) and the Benchmark tab. Lighting, fog and bloom are not among the four required algorithms.

Chapter 2: MATHEMATICAL BACKGROUND

Points are column vectors and matrices are stored row by row. The camera looks down the negative z axis.

2.1 Outer Product and Area

For two vectors $\mathbf{u}, \mathbf{v}$ in the plane, the outer product $\mathbf{u} \wedge \mathbf{v}$ is a parallelogram with a direction. It has one number, $u_x v_y - u_y v_x$, and the size of that number is the area of the parallelogram. A triangle with corners a, b, c has area $\frac{1}{2}|(b-a) \wedge (c-a)|$. A positive sign means c is to the left of the line from a to b (anticlockwise). A negative sign means clockwise. Zero means the three points are on one line.

In 3D the outer product has three numbers, the same three as the cross product $\mathbf{u} \times \mathbf{v}$. The cross product writes them as an arrow at right angles to both vectors, and the length of the arrow is the parallelogram's area. We use this for triangle normals in culling and the 2D version in Visvalingam–Whyatt.

2.2 Transformations and Determinants

We write a 2D point as $(x, y, 1)^T$ and a 3D point as $(x, y, z, 1)^T$, so that moving a point is also a matrix. Rotation about the origin by θ in 2D is

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

To rotate about a point p, we move p to the origin, rotate, and move back: $T(p)\, R(\theta)\, T(-p)$. In 3D, rotation about an axis uses Rodrigues' formula.

The determinant says how area (2D) or volume (3D) changes. A rotation has determinant 1. A shear also has determinant 1 (area stays, shape changes). A scale by (s_x, s_y) has determinant $s_x s_y$. A reflection has determinant -1, which means the direction of turning is reversed. If the determinant is 0, the shape is flattened.

2.3 Quaternions

A unit quaternion $q = (\cos\frac{\theta}{2},\ \sin\frac{\theta}{2}\,\hat{\mathbf{n}})$ is a rotation by θ about the unit axis $\hat{\mathbf{n}}$. Two rotations are combined by multiplying their quaternions. Euler angles rotate about x, y and z one after the other. When the middle angle is 90°, two of the axes line up and one way of turning is lost. This is called gimbal lock. To move smoothly from q_0 to q_1 we use slerp [1]:

$$\operatorname{slerp}(q_0, q_1, t) = \frac{\sin((1-t)\Omega)}{\sin\Omega}\, q_0 + \frac{\sin(t\Omega)}{\sin\Omega}\, q_1, \qquad \cos\Omega = q_0 \cdot q_1.$$

Slerp turns at a constant speed. Plain linear interpolation (lerp) followed by normalizing follows the same path but is faster in the middle.

2.4 Bases and Handedness

A set of vectors spans a space if every vector in the space can be made from them. The rank is the number of independent vectors. Gram–Schmidt turns independent vectors into perpendicular ones. For three vectors $\mathbf{a}, \mathbf{b}, \mathbf{c}$, the triple product $\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$ is the signed volume of the box they make. Positive means a right-handed system, negative means left-handed, and zero means the vectors are flat. The sine rule says $a/\sin A = b/\sin B = c/\sin C = 2R$, where R is the radius of the circle through the three corners.

Chapter 3: DESIGN AND ARCHITECTURE

3.1 Tools

RenderLab is written in TypeScript. The interface uses React 19 and is built with Vite. Tests run with Vitest [2]. The maths, line drawing, triangle filling and blending are our own code. We did not use a graphics or maths package.

3.2 Two Layers

The code is in two folders (Figure 3.1). The folder `src/core` holds the algorithms as plain functions. They take numbers and return numbers, and they do not use React or the browser, so they can be tested without a screen. The folder `src/ui` has one React file for each lab and the editor window. A lab calls the core, writes the result into a framebuffer, and shows it on a canvas with `putImageData`.

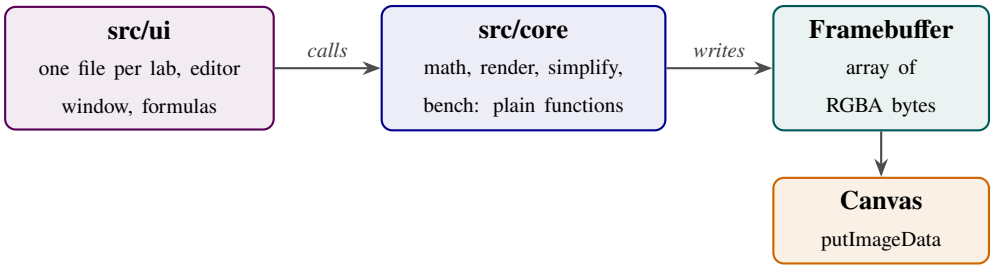


Figure 3.1: The two layers of RenderLab and the path of one frame

3.3 Drawing Steps for One Frame

1. Each vertex is moved by the model matrix into world space.
2. Back-facing triangles are removed.
3. The view and perspective matrices give each vertex a pixel position. The *y* axis is flipped because screen rows grow downward. If a vertex is behind the camera, the triangle is skipped (there is no clipping).
4. Each solid triangle is filled using a depth buffer. See-through triangles are drawn in a second pass.
5. Wireframes and outlines are drawn with Bresenham's line.

The `Framebuffer` class stores four bytes per pixel, row by row from the top. Because it is a `Uint8ClampedArray`, any value above 255 or below 0 is clamped by the language. The depth buffer keeps one number per pixel: the smallest depth drawn so far.

3.4 The Editor Window

All labs share one window. A lab gives the window a canvas, some controls, a stats line and some lines for the Output panel. Equations are written in a small plain-text form and drawn by our own parser (`mathParse.ts`), so the formulas on screen use the same numbers as the code.

Chapter 4: THE FOUR REQUIRED ALGORITHMS

4.1 Bresenham Line Drawing

A pixel grid has whole-number positions, but a real line passes between them. For each step along the line we must pick the pixel nearest to the line [3]. Call the longer direction the major axis and the shorter one the minor axis, with lengths Δ_M and Δ_m. We move one pixel at a time along the major axis. At each step the minor coordinate either stays the same (a straight step) or changes by one (a diagonal step). A whole-number error value decides which:

$$e_0 = 2\Delta_m - \Delta_M, \qquad e_{i+1} = \begin{cases} e_i + 2(\Delta_m - \Delta_M) & e_i > 0 \ \text{(diagonal)} \\ e_i + 2\Delta_m & e_i \le 0 \ \text{(straight)}. \end{cases}$$

Only whole-number additions and comparisons are needed, with no division and no rounding. When $e = 0$ the line is exactly half way between two pixels. We then pick the larger minor coordinate, so a line drawn backwards gives the same pixels as one drawn forwards.

```
const diagonal = error > 0 || (error === 0 && minorSign > 0)
if (diagonal) { error += 2 * (minor - major); x += sx; y += sy }
else          { error += 2 * minor; if (steep) y += sy; else x += sx }
```

Figure 4.1: The decision in `bresenhamSteps` (`src/core/render/line.ts`)

Worked example. For the line from $(0, 0)$ to $(5, 2)$ we have $\Delta_M = 5$, $\Delta_m = 2$ and $e_0 = -1$ (Table 4.1). The unit test checks exactly these values. We also kept the older DDA method in the code, only so the lab can draw it over Bresenham's line (Figure 4.2).

Table 4.1: Bresenham steps for the line $(0, 0) \rightarrow (5, 2)$

Pixel	Error before	Move	Error after
(0,0)	−1	straight	3
(1,0)	3	diagonal	−3
(2,1)	−3	straight	1
(3,1)	1	diagonal	−5
(4,2)	−5	straight	−1
(5,2)	−1	none	−1

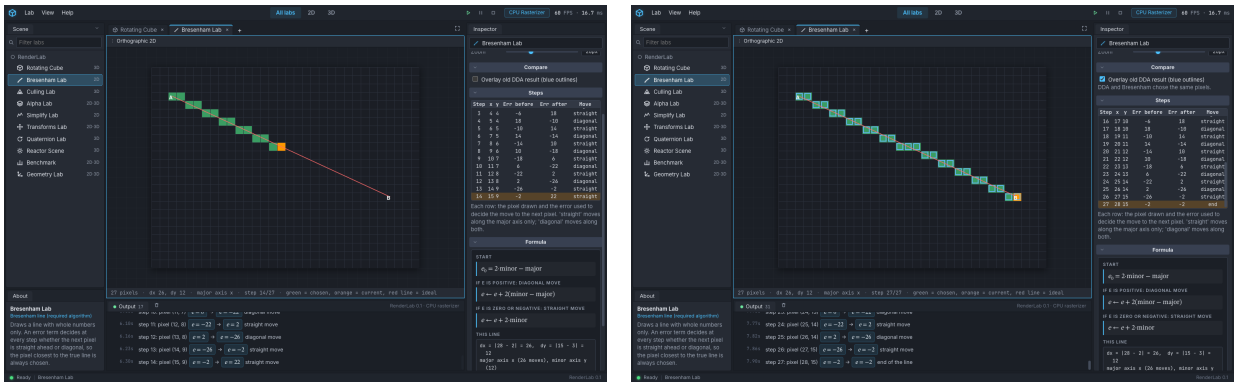


Figure 4.2: Bresenham Lab. Left: a line half drawn with its table of error values. Right: the DDA line drawn over it.

4.2 Back-Face Culling

On a closed solid, a triangle that faces away from the camera is always hidden behind the triangles that face it, so we can skip it. The front of a triangle is the side from which its corners $a \to b \to c$ go anticlockwise. With $e_1 = b - a$ and $e_2 = c - a$, the normal is $\mathbf{n} = e_1 \times e_2$. Its length is twice the triangle's area, which is the length of the outer product $e_1 \wedge e_2$. The vector from the eye E to the triangle is $a - E$, and the triangle is back-facing when

$$\mathbf{n} \cdot (a - E) \;\geq\; 0.$$

The lab also has a second test that uses the projected corners. The doubled signed area $(x_1 - x_0)(y_2 - y_0) - (y_1 - y_0)(x_2 - x_0)$ is negative for a front-facing triangle, because screen y points down. Both tests give the same answer. A triangle seen exactly edge-on gives 0 in exact maths, but rounding can give a tiny value like 10^{-17} of either sign. Both tests treat anything closer to zero than 10^{-9} as back-facing, so they always agree.

Worked example. Let $a = (0, 0, 0)$, $b = (1, 0, 0)$, $c = (0, 1, 0)$ and the eye $E = (0, 0, 5)$. Then $\mathbf{n} = (0, 0, 1)$, the area is 0.5, and $\mathbf{n} \cdot (a - E) = -5$. This is below zero, so the triangle is front-facing. If we list the corners as a, c, b, the normal becomes $(0, 0, -1)$ and the product is +5, so the triangle is back-facing. Our tests check both orders. About half of the triangles of a closed shape face away (Figure 4.3).

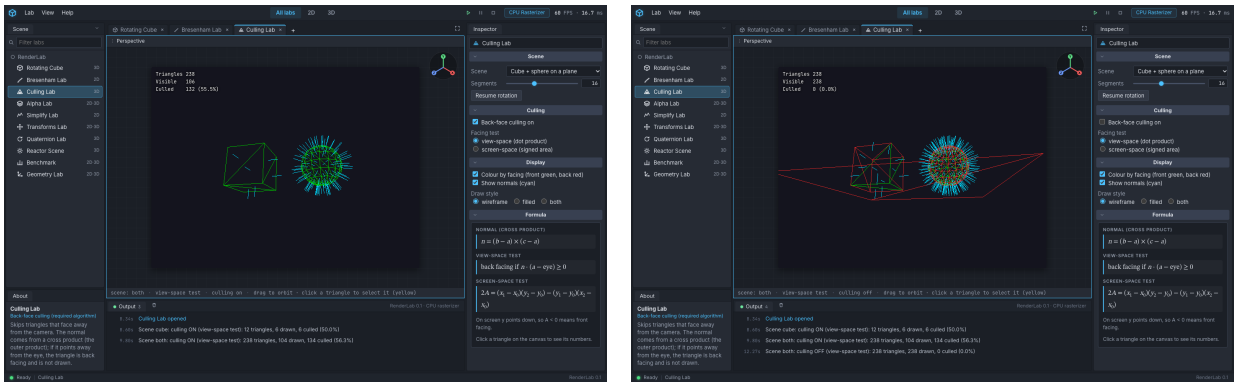


Figure 4.3: Culling Lab, wireframe with normals. Left: culling on. Right: culling off, so the back faces are drawn too.

4.3 Alpha Compositing

A colour with alpha has red, green and blue from 0 to 255 and an alpha from 0 (invisible) to 1 (solid). The "over" rule says what we see when a see-through colour (the source) is laid over another colour (the destination) [4]. The source hides a share α_s of what is below, so a share $1 - \alpha_s$ of the destination still shows:

$$\alpha_o = \alpha_s + \alpha_d(1 - \alpha_s), \qquad C_o = \frac{C_s\alpha_s + C_d\alpha_d(1 - \alpha_s)}{\alpha_o}.$$

We divide by α_o to turn the weighted sum back into a normal colour. If $\alpha_o = 0$ we return transparent black, to avoid dividing by zero. The code also has a premultiplied version, where every channel uses one rule: $X_o = X_s + X_d(1 - \alpha_s)$. A test checks that both versions give the same picture.

```
const outA = as + ad * (1 - as)
if (outA === 0) return [0, 0, 0, 0]
const dstWeight = ad * (1 - as)
return [(src[0]*as + dst[0]*dstWeight) / outA, /* same for g and b */ ..., outA]
```

Figure 4.4: The core of `over` (`src/core/render/alpha.ts`)

Worked example. A half-transparent red $(255, 0, 0, 0.5)$ over a solid blue $(0, 0, 255, 1)$ gives $\alpha_o = 1$

and the colour $(127.5, 0, 127.5)$, a purple. A solid blue over the red gives pure blue, because a solid source hides everything. So the order of the layers matters.

For many see-through triangles in 3D, the renderer draws in two passes. Solid triangles come first and write the depth buffer. See-through triangles come second. They use the depth test (so a pane behind a solid object is hidden) but do not write depth (so panes do not hide each other). They are sorted from far to near, so the farthest is blended first. The lab can switch the sort off, which gives wrong colours (Figure 4.5).

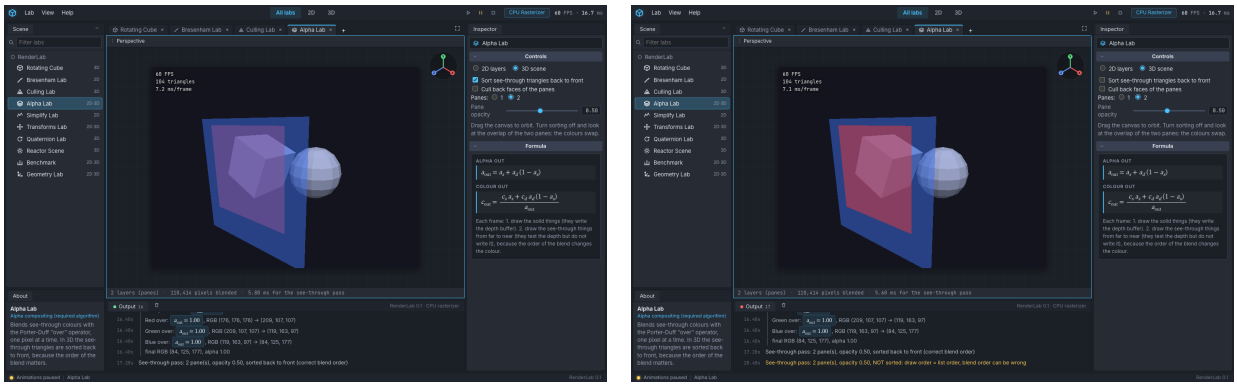


Figure 4.5: Alpha Lab in 3D. Left: panes sorted back to front. Right: sorting off, so the colours are wrong.

4.4 Visvalingam–Whyatt Simplification

A polyline with many points has points that hardly bend it. For each point we look at the triangle it makes with its two neighbours. A small triangle means the point is not needed. We remove the point with the smallest triangle, recompute the triangles of its two neighbours, and repeat until enough points are left [5]. The area of a triangle is half the 2D outer product of two of its edges.

The two end points of an open line are never removed. A closed polygon keeps at least three points. If a neighbour's new triangle is smaller than the one just removed, we rank it with the removed area instead: ranking area = $\max(\text{new area},\ \text{last removed area})$. This keeps the removal areas from ever going down, so "remove all points below area X" is the same as "remove the first k points". Ties go to the lower index.

Our version scans all points for the smallest area after every removal. This costs $O(n^2)$ for n points. A heap would give $O(n \log n)$, but the scan is short and easy to explain, and Chapter 7 measures its cost.

```
for (let i = 0; i < n; i++)
  if (alive[i] && canGo(i) && (best < 0 || area[i] < area[best])) best = i
...
area[v] = Math.max(rawArea[v], lastArea)   // the rule that areas never go down
```

Figure 4.6: The scan and the ranking rule (`src/core/simplify/visvalingam.ts`)

Worked example. Take the open line $(0,0), (1,1), (2,0), (3,3)$. Point 1 makes the triangle $(0,0), (1,1), (2,0)$ with area 1. Point 2 makes $(1,1), (2,0), (3,3)$ with area 2. Point 1 goes first, with area 1. Point 2 now has the neighbours $(0,0)$ and $(3,3)$, a triangle of area 3, and it is ranked $\max(3,1) = 3$. Our function returns these two removals with areas 1 and 3. We also kept Ramer–Douglas–Peucker [6, 7] for comparison (Figure 4.7).

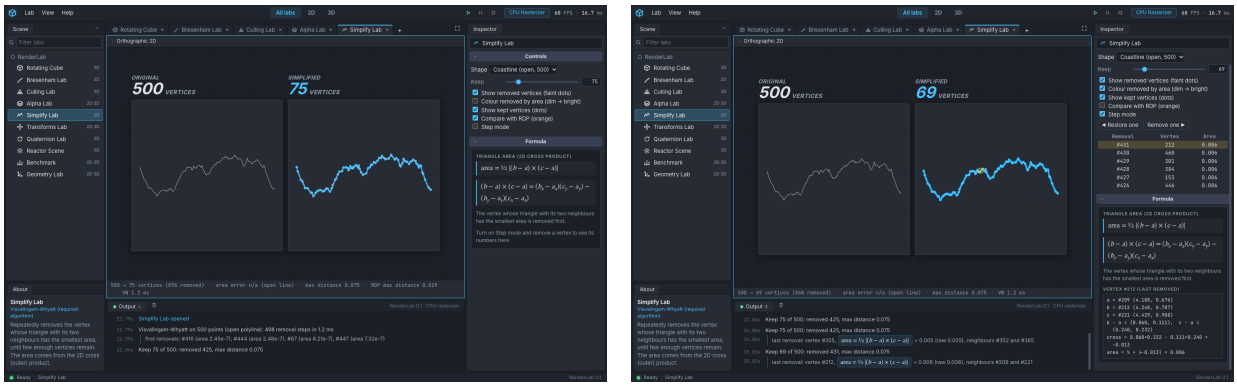


Figure 4.7: Simplify Lab. Left: 500 points against 75, with the RDP result drawn over it. Right: step mode, one removal at a time.

Chapter 5: THE OTHER LABS AND THE REACTOR SCENE

5.1 Transforms Lab

This lab applies a list of operations to a letter F (2D) or a cube (3D). In 2D the operations are translate, rotate, scale, shear, reflect and rotate about a point. In 3D they are translate, rotate about x, y or z, rotate about any axis, rotate about an axis through a point, scale and shear. The operations can be added, removed and reordered. The lab multiplies their matrices in the order given. It shows the combined matrix, the two basis arrows, and the determinant with its meaning (how area or volume changes, and whether the direction of turning is reversed). It also names the result, for example rotation, reflection or shear. Changing the order of a rotation and a translation shows that matrix products depend on order.

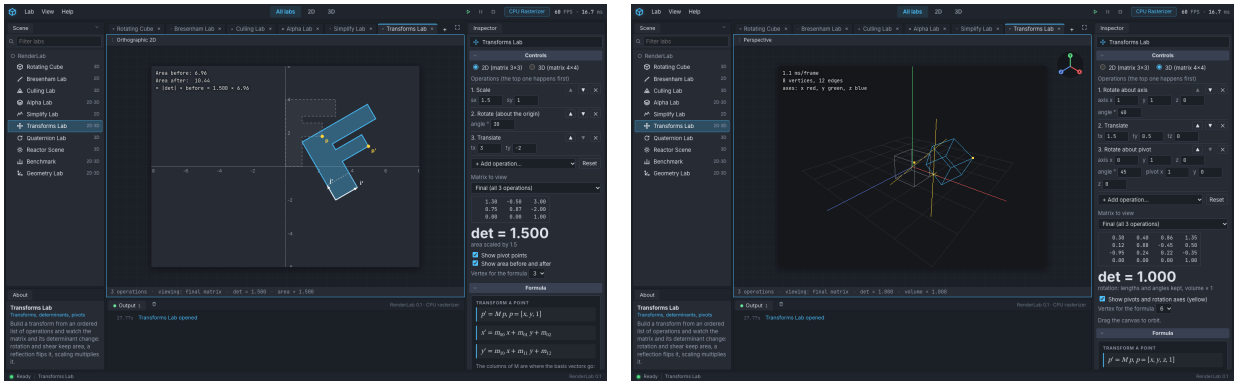


Figure 5.1: Transforms Lab. Left: 2D, with the F shape, basis arrows and determinant. Right: 3D.

5.2 Quaternion Lab

The lab has two modes. In the first, two copies of a model are turned by the same yaw, pitch and roll, one with Euler matrices and one with a quaternion. At pitch 90° a gimbal lock badge appears and the lab says which two axes became the same line. The quaternion side keeps turning. In the second mode, the lab moves between two rotations with slerp and with lerp, and plots the angle turned at each step. Slerp gives a flat line, and lerp gives a curve (Figure 5.2).

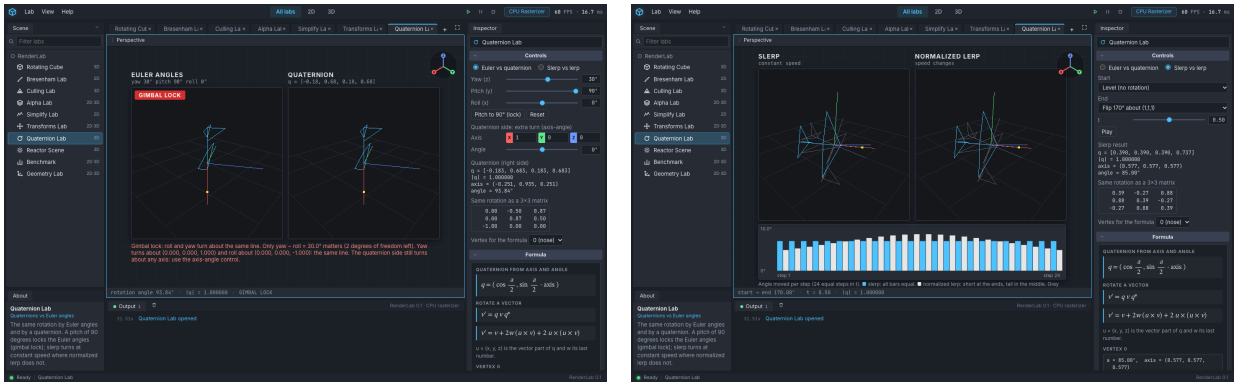


Figure 5.2: Quaternion Lab. Left: gimbal lock at pitch 90°. Right: slerp against lerp with the plot.

5.3 Geometry Lab

The Geometry Lab has four modes, and the points can be dragged with the mouse.

- **Outer product:** the area of the parallelogram and the triangle, clockwise or anticlockwise, and a comparison with the 3D cross product.
- **Sine rule:** the sides, the angles, the three ratios, and the circle through the corners.

- **Line intersection:** the meeting point of two lines, or a message that they are parallel.
- **Bases:** rank, whether the vectors span the plane, coordinates in a basis, Gram–Schmidt, and in 3D the triple product with right-handed or left-handed axes.

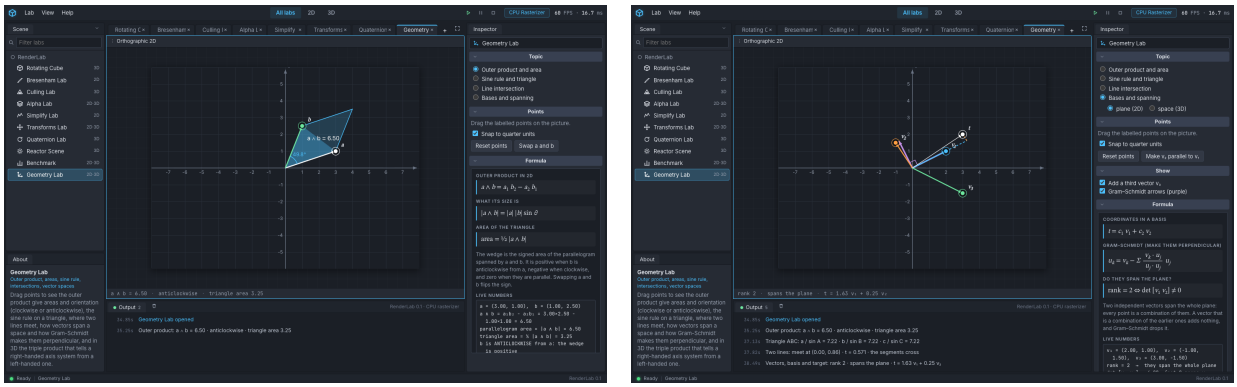


Figure 5.3: Geometry Lab. Left: outer product mode. Right: bases in the plane with Gram–Schmidt.

5.4 Reactor Scene and Scene Editor

The Reactor Scene is an extra, and it is not one of the four required algorithms. It has 271 objects and 15,412 triangles in six folders. Every triangle goes through the same culling, projection and depth test as in the smaller labs. A frame is drawn in this order: a sky gradient; the solid triangles, each lit once from its normal and mixed with fog by its distance; the grid lines, drawn with Bresenham; and last a bloom glow made by blurring the brightest pixels. Lighting, fog and bloom are extras.

The scene editor lets the user click an object in the picture or in the Scene tree, change its name, position, rotation, scale and colour, add a cube, sphere, cylinder or plane, and delete the selected object. The model matrix is built as $T\,R_z R_y R_x\,S$. To pick an object, the editor asks every object which of its triangles is under the clicked pixel (the same picking as in the Culling Lab), and the nearest hit wins. The selected object is outlined with the edges where a triangle facing the camera meets one facing away, drawn with Bresenham's line. The picture can be saved as a PNG or copied.

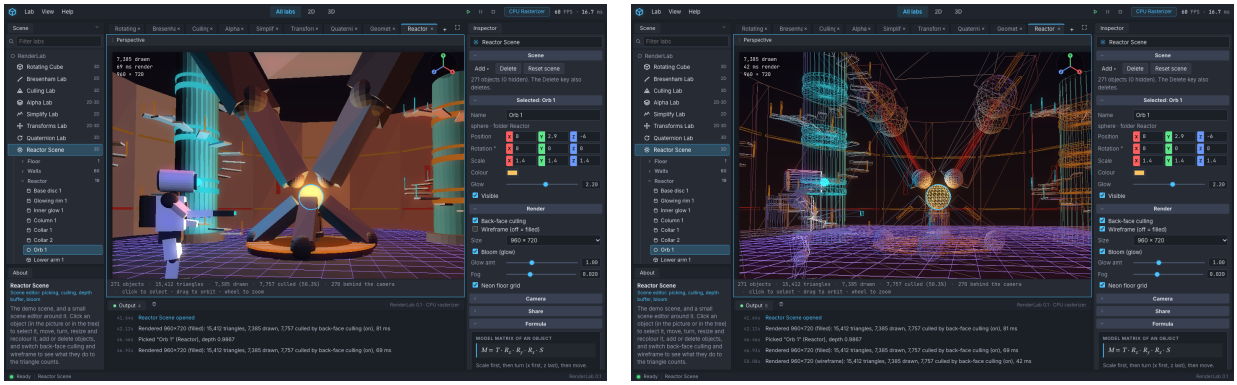


Figure 5.4: Reactor Scene. Left: one object selected and outlined. Right: the same scene as a wireframe.

Chapter 6: TESTING

Every function in `src/core` has a test file. We run them with `npm test`. The final run gave 31 test files and 487 tests, all passing, and `npx tsc --noEmit` showed no type errors. The interface code is not unit tested, except the formula parser, because it has no algorithms. Instead we tried every control in a real browser.

Table 6.1: Tests by area

Area	Tests
Maths (vectors, matrices, quaternions, and the helpers of three labs)	159
Rendering (Bresenham, culling, alpha, camera, fill, meshes, lighting, scene, editor)	229
Line simplification (Visvalingam–Whyatt, RDP, error measures, shapes)	54
Benchmark tools	36
Formula parser	9
Total (31 files)	487

Bresenham has 42 tests (and 3 for DDA). They check hand-worked lines with every error value, all eight directions, and every pair of end points with coordinates from −4 to 4 (6,561 lines). For each line they check the pixel count, the end points, that no pixel is more than half a pixel from the true line, that neighbouring pixels touch, and that the reverse line has the same pixels. **Culling** has 29 tests for normals, areas, the two tests agreeing, and edge-on triangles. **Alpha** has 25 tests for hand-worked blends, the order of layers, and the premultiplied version. **Visvalingam–Whyatt** has 33 tests (54 with RDP, error measures and shapes) for areas, the hand-worked removal order, end points staying, and removal areas never going down.

Click-through. We used every control of every lab in a real browser at 1440 by 900 pixels, and opened every lab again at 1280 by 720 to check the layout. All 195 checks pass. The table is in `docs/QA.md`. We found two layout bugs and fixed them: tab labels wrapped to two lines and covered the picture, and the caption of the slerp plot was cut off. Neither was in an algorithm.

Chapter 7: EXPERIMENTAL RESULTS

7.1 How We Measured

The numbers come from the Benchmark tab of RenderLab. We ran all five suites once, with 15 timed runs after 5 warm-up runs for every row. The tab reports the median of the timed runs. The exported files are in `report/data/`, and every number below is copied from them.

- Computer: a Mac with 10 CPU cores. Browser: headless Chrome 156, driven by a script so the page stayed in focus. A normal browser may give different times.
- Each frame is 640 by 480 pixels. We ran nothing else during the test.
- This is one run on one machine. Small differences (a few percent) are not reliable. The comparisons between rows are more useful than the exact milliseconds.

7.2 Back-Face Culling

In the demo scene, culling removed 7,757 of the 15,412 triangles (50.33%). The filled frame took 41.4 ms with culling off and 28.0 ms with it on, which is 1.48 times faster (Table 7.1). The saving is smaller than the share of culled triangles. Culling still has to move every vertex and adds one test for each triangle. We did not time these parts separately. For the small spheres a frame takes only 1 to 2 ms, and the wireframe differences there (for example 1.125 ms against 1.075 ms) are as small as the noise in the run. We draw no conclusion from those rows.

Table 7.1: Frame time with culling off and on (median of 15 runs, ms)

		Filled			Wireframe	
Scene	Culled %	Off	On	Faster by	Off / on	Faster by
Reactor scene	50.33	41.4	28	1.48	41.4 / 29.1	1.42
Sphere, 8 segments	54.17	2.1	1.662	1.26	1.125 / 1.075	1.05
Sphere, 16 segments	58.04	2.25	1.638	1.37	1.387 / 1.188	1.17
Sphere, 32 segments	62.08	2.975	2	1.49	2.175 / 1.787	1.22
Sphere, 64 segments	62.3	5.25	3.425	1.53	4.6 / 3	1.53

7.3 Bresenham against DDA

We drew 10,000 random lines of three lengths (Table 7.2). Both methods made the same number of pixels, in the same places. DDA was faster in every row, about 1.8 to 2.1 times when drawing. The textbook says Bresenham is faster because it avoids division and decimals, but JavaScript has no separate whole-number type, so every number is already a decimal number and the saving does not appear. Also, our `bresenhamSteps` stores the error before and after each step so the lab can show a table, so it builds a bigger object for each pixel than DDA does. We did not measure how much of the gap comes from this. The speed benefit of Bresenham is a property of integer hardware, not of this program.

Table 7.2: Time for 10,000 lines (median, ms)

Method	Short (10 px)	Medium (100 px)	Long (500 px)
Bresenham `drawLine`	3.2	26.5	122.5
DDA with `setPixel`	1.512	12.8	68
`bresenhamSteps` only	1.525	13	83.4
`ddaSteps` only	0.7438	5.9	29.2

7.4 Alpha Compositing

We applied `over()` to every pixel of a 640 by 480 image with 1, 2, 4 and 8 layers. The time grows with the number of layers: 4.3 ms for one layer and 25.1 ms for eight, which is 10 to 14 ns per call. For the 3D see-through pass with 6 overlapping panes (432 triangles), the time was 30.4 ms without the sort and 30.8 ms with it. The sort costs about 1.3%, and it makes the colours correct.

7.5 Visvalingam–Whyatt against RDP

We simplified a seeded coastline of 100 to 2,000 points to 10% of its points (Table 7.3). When the number of points doubles, the time of VW grows by about 4 times (4.03 from 500 to 1,000 points and 3.97 from 1,000 to 2,000), which is what $O(n^2)$ predicts. At 2,000 points VW takes 14.3 ms and RDP takes 0.42 ms, so VW is about 34 times slower. This is the cost of our scan for the smallest triangle. For the shape, the two are close. At 500 points the area error is 0.23% for VW and 0.72% for RDP, but at 200 points RDP is better (0.31% against 1.95%). The largest distance from an original point to the new line was smaller for RDP at every size. So we do not say one method is better on this data.

Table 7.3: Simplifying to 10% of the points (median ms per call)

Points	VW (ms)	VW time ratio	n^2 ratio	RDP (ms)	VW area error %
100	0.04609			0.009668	8.128
200	0.1508	3.271	4	0.025	1.954
500	0.8938	5.927	6.25	0.07578	0.2306
1000	3.6	4.028	4	0.1734	0.0261
2000	14.3	3.972	4	0.4156	0.02659

7.6 Transformations

For one million operations each, a `mat4` multiply took 156 ns and an inverse took 1,047 ns. Turning a point with a quaternion took 12.8 ns and with a matrix 8.6 ns. For these last two rows the minimum (6.5 and 3.7 ns) is far below the median, so they vary between runs. We only say that both are very cheap.

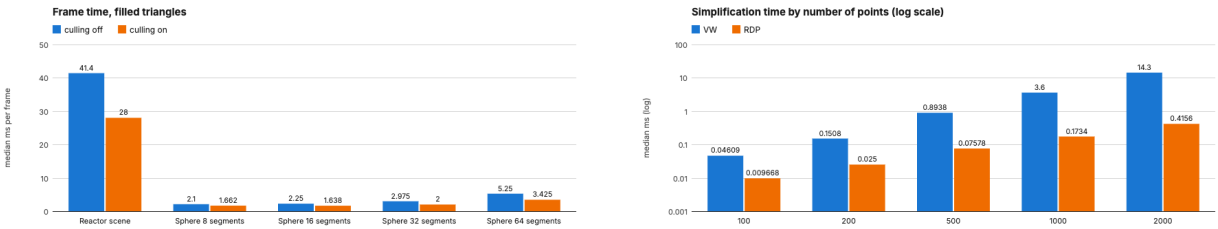


Figure 7.1: Charts saved from the Benchmark tab. Left: frame time with culling off and on. Right: time to simplify, VW against RDP.

Chapter 8: LIMITATIONS, CONCLUSION AND FUTURE WORK

8.1 Limitations

- **No clipping.** A triangle with a corner behind the camera is skipped completely, not cut. In the demo scene this skipped 359 of 15,412 triangles when culling was off.
- **Flat shading.** Each triangle has one colour, so curved objects show their flat faces.
- **One-sided planes.** A plane has one front side and disappears when seen from behind with culling on.
- **Visvalingam–Whyatt is $O(n^2)$.** It is fine for the lab but would be slow for millions of points.
- **One run, one machine.** The benchmark was run once in headless Chrome. The relative results are useful, but the exact milliseconds will differ on other computers.
- **Bresenham is not faster in JavaScript,** as the benchmark shows.
- **The interface has no automated tests.** We checked it by hand, in Chrome on macOS only.
- **Lighting, fog and bloom** are simple extras and are not among the four required algorithms.

8.2 Conclusion

RenderLab has the four required algorithms (Bresenham, back-face culling, alpha compositing and Visvalingam–Whyatt) as plain TypeScript with no graphics library. Each has a lab that shows the formulas, the numbers at every step, and a comparison. Other labs cover transformations, determinants, quaternions, the outer product, the sine rule, line intersection, bases and handedness. All 487 tests pass, and all 195 controls passed our click-through after two layout fixes. The benchmark gave some results we did not expect: DDA was faster than Bresenham in JavaScript, and Visvalingam–Whyatt was about 34 times slower than RDP at 2,000 points.

Table 8.1: Contribution of each student

Name	Work
Yasharth Singh	TODO: fill in before submission
Rakshit Suneja	TODO: fill in before submission

8.3 Future Work

- Port the core to Rust and WebAssembly. Bresenham's whole-number arithmetic should then show the speed benefit that JavaScript does not give, and the benchmark can be repeated to check.
- Use a heap in Visvalingam–Whyatt to make it $O(n \log n)$.
- Cut triangles at the near plane (clipping) so they do not disappear close to the camera.
- Add smooth shading and textures.
- Add automated tests for the interface.

REFERENCES

[1] K. Shoemake, “Animating rotation with quaternion curves,” in *Proceedings of SIGGRAPH ’85, Computer Graphics*, vol. 19, no. 3, 1985, pp. 245–254.

[2] Vitest contributors, “Vitest: a Vite-native test framework,” https://vitest.dev.

[3] J. E. Bresenham, “Algorithm for computer control of a digital plotter,” *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965.

[4] T. Porter and T. Duff, “Compositing digital images,” in *Proceedings of SIGGRAPH ’84, Computer Graphics*, vol. 18, no. 3, 1984, pp. 253–259.

[5] M. Visvalingam and J. D. Whyatt, “Line generalisation by repeated elimination of points,” *The Cartographic Journal*, vol. 30, no. 1, pp. 46–51, 1993.

[6] U. Ramer, “An iterative procedure for the polygonal approximation of plane curves,” *Computer Graphics and Image Processing*, vol. 1, no. 3, pp. 244–256, 1972.

[7] D. H. Douglas and T. K. Peucker, “Algorithms for the reduction of the number of points required to represent a digitized line or its caricature,” *Cartographica: The International Journal for Geographic Information and Geovisualization*, vol. 10, no. 2, pp. 112–122, 1973.